



DR B. DIOP

babacar.diop@ussein.edu.sn

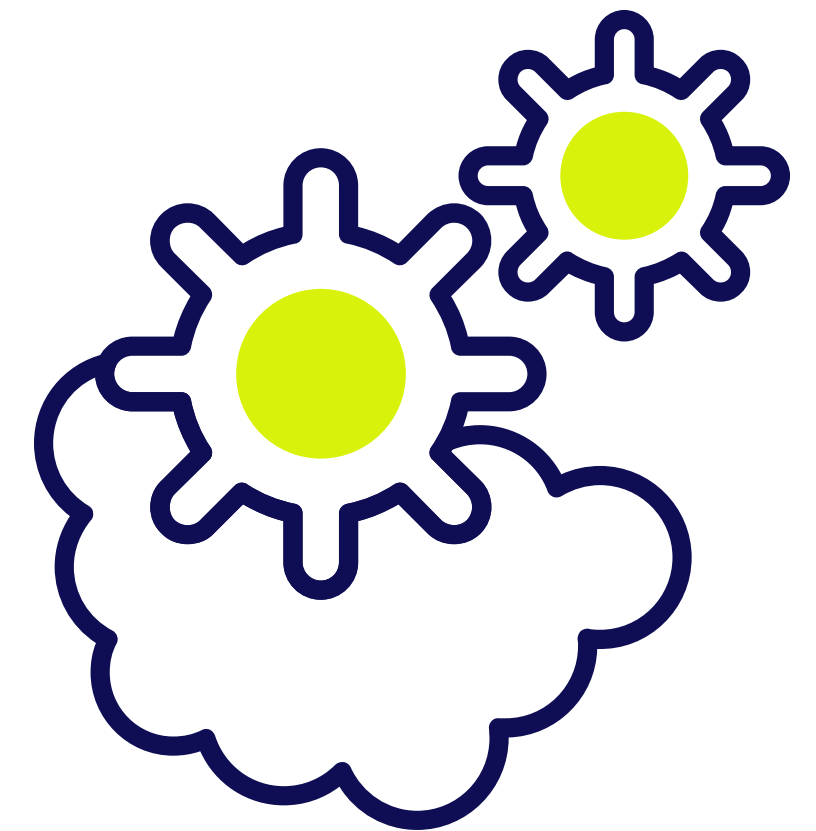
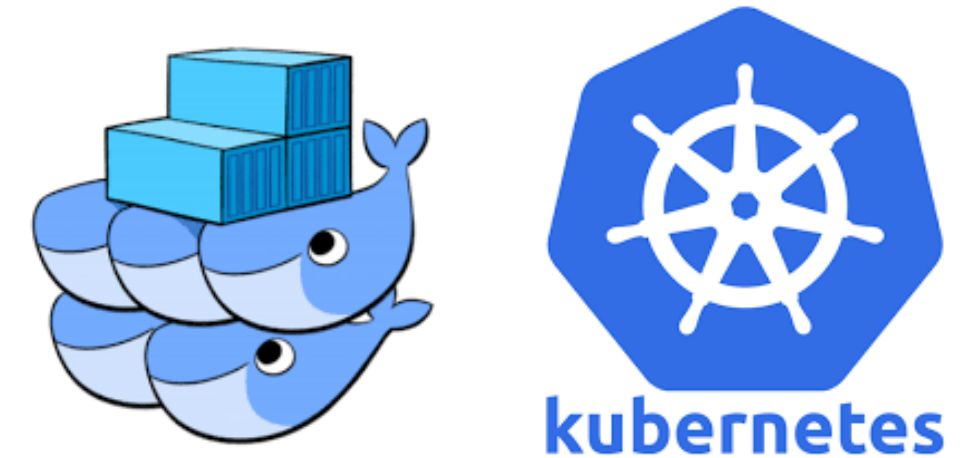
Conteneurisation & orchestration

Kubernetes Orchestration

Services

INFO – AGROTIC

Last update: dec. 2025



Problématique de la Connexion Directe aux Pods

Issues : 4 problèmes liés aux **Pods** :

1. En cas d'échec, un Pod est remplacé → nouvelle adresse IP.
2. La montée en charge (scale-up) ajoute des Pods → nouvelles IP.
3. La réduction (scale-down) supprime des Pods.
4. Les mises à jour progressives (rolling updates) remplacent aussi les Pods.

Conséquence : une instabilité permanente des adresses IP.

Problématique de la Connexion Directe aux Pods

Issues : 4 problèmes liés aux **Pods** :

1. En cas d'échec, un Pod est remplacé → nouvelle adresse IP.
2. La montée en charge (scale-up) ajoute des Pods → nouvelles IP.
3. La réduction (scale-down) supprime des Pods.
4. Les mises à jour progressives (rolling updates) remplacent aussi les Pods.

Conséquence : une instabilité permanente des adresses IP.

Règle essentielle : Ne jamais se connecter directement à un Pod spécifique.

Les Services - Principes fondamentaux

1. Le service

- Le Service (avec un "S") est un objet API Kubernetes.
- Il offre une connectivité réseau stable pour un groupe de **Pods**.
- Défini par un manifeste **YAML**, comme un **Pod** ou un **Deployment**.

Les Services - Principes fondamentaux

2. Des Points Stables de Connexion

Chaque Service fournit :

-  Une IP fixe (clusterIP)
-  Un nom DNS stable
-  Un port stable

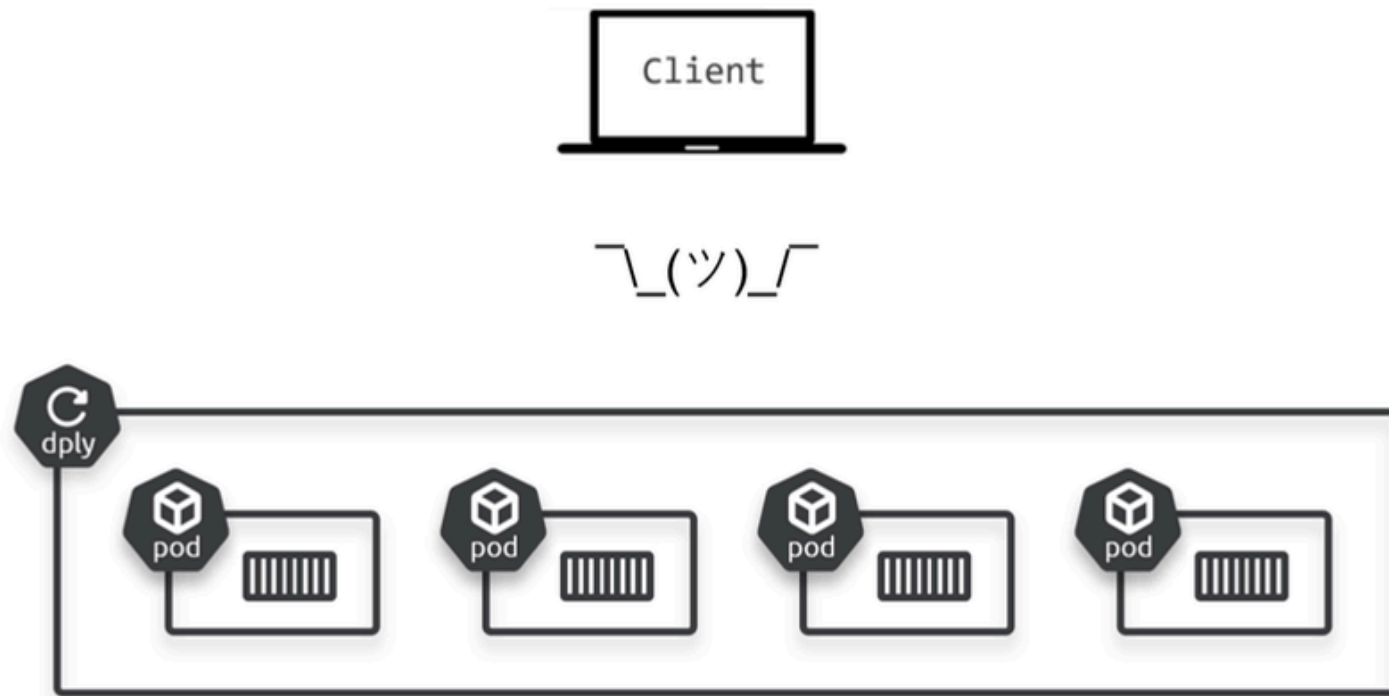
Les Services - Principes fondamentaux

3. Sélection Dynamique des Pods

- Le **Service** utilise un système de labels et de selecteurs.
- Il route automatiquement le trafic vers tous les **Pods** correspondants.
- Flexible et dynamique : les **Pods** peuvent changer, le **Service** reste.

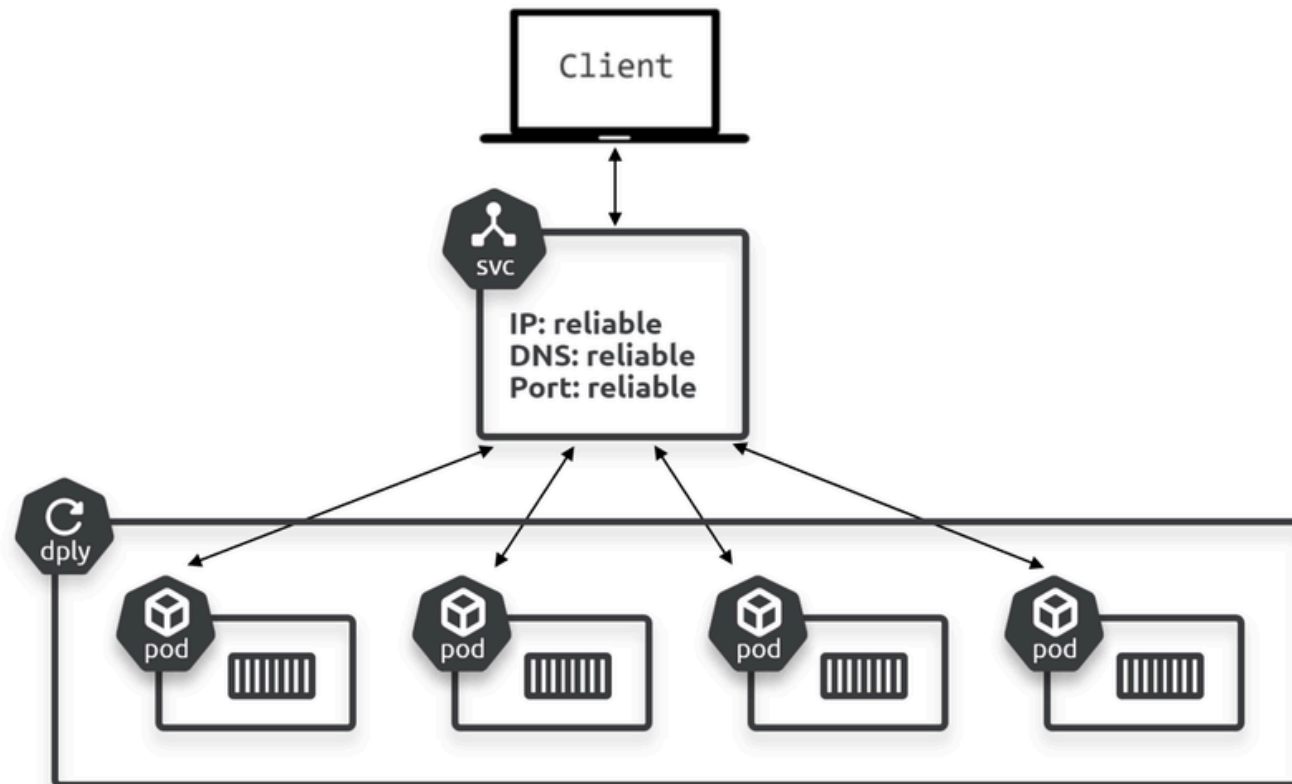
Les Services - Besoin client en réseau

La figure ci-dessous illustre une application simple gérée par un contrôleur Deployment. Un client (qui peut être un autre microservice) a besoin d'un point de terminaison réseau fiable pour accéder aux Pods.



Les Services - Solution pour le client

Cette figure montre la même application avec l'ajout d'un Service. Le Service expose les Pods avec une IP, un nom DNS et un port stables. Il répartit également la charge du trafic vers les Pods ayant les bons labels.



Les Services - Abstraction totale

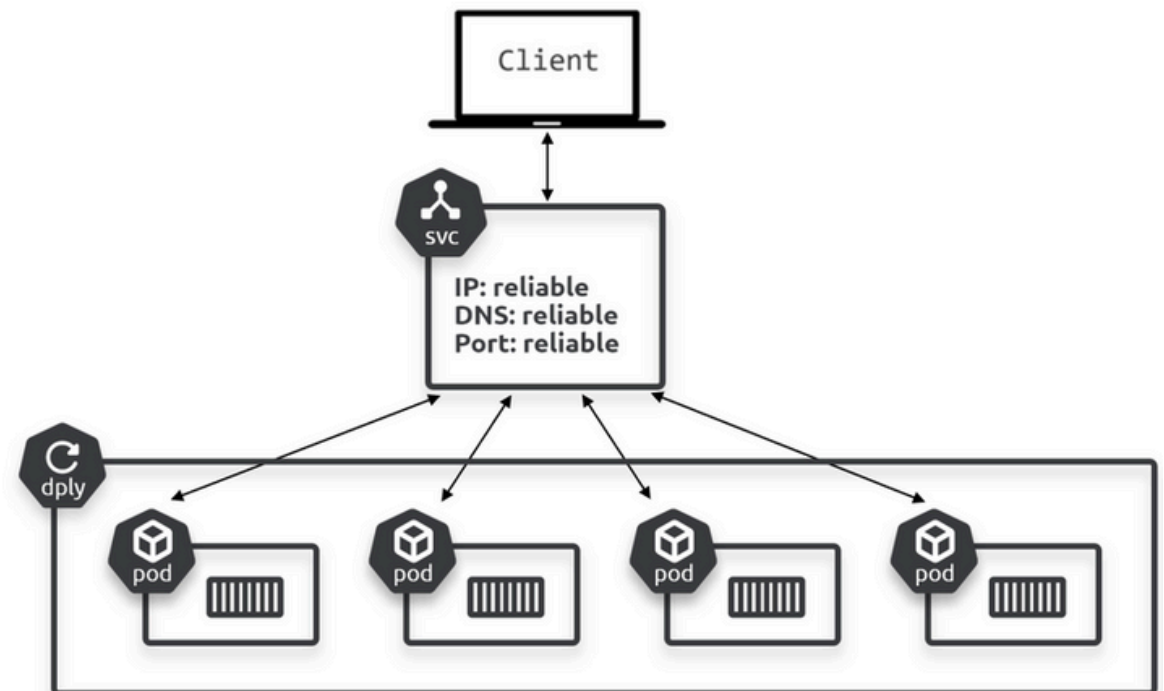
Cette figure montre la même application avec l'ajout d'un Service. Le Service expose les Pods avec une IP, un nom DNS et un port stables. Il répartit également la charge du trafic vers les Pods ayant les bons labels.

Le Service, une Abstraction Stable

Avec un Service en place :

- Les Pods peuvent monter/descendre en charge
- Ils peuvent tomber en panne
- Ils peuvent être mis à jour ou restaurés

Résultat : Les clients accèdent à l'application sans interruption.



Les Services - Abstraction et fonctionnement

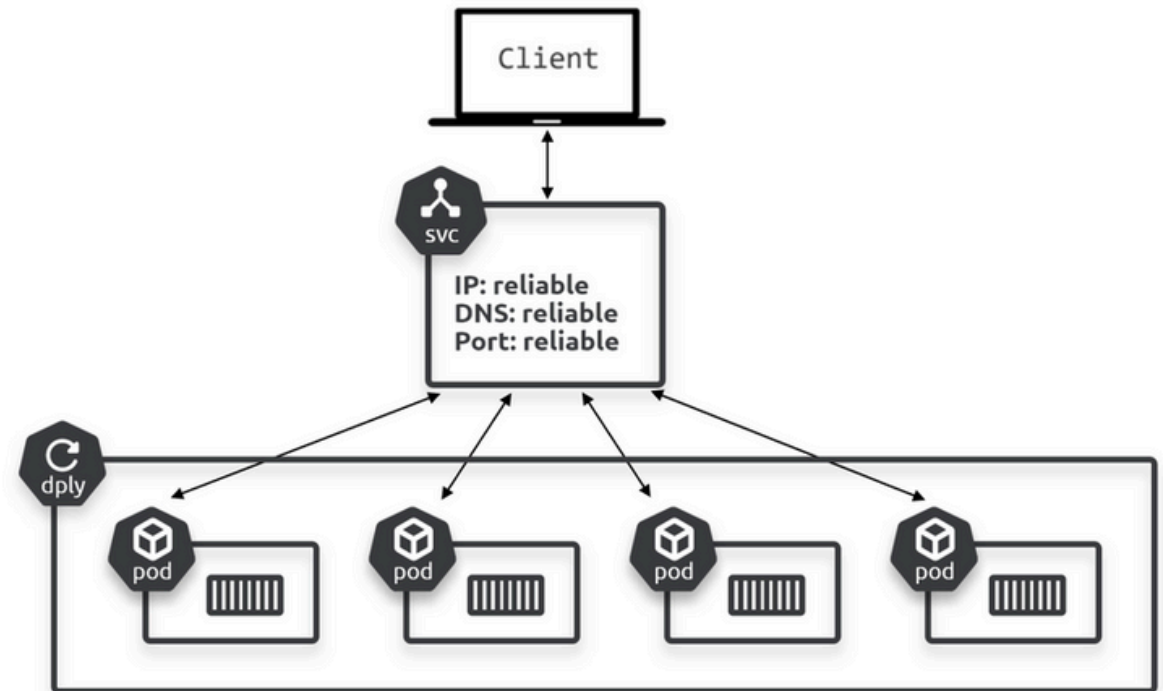
Cette figure montre la même application avec l'ajout d'un Service. Le Service expose les Pods avec une IP, un nom DNS et un port stables. Il répartit également la charge du trafic vers les Pods ayant les bons labels.

Comment ça fonctionne ?

Le Service observe les changements et met à jour sa liste de Pods sains en temps réel.

Mais il ne modifie jamais :

- ☒ Son IP stable
- ☒ Son nom DNS
- ☒ Son port



Étiquettes et Couplage Lâche

Principe clé

Les **Services** sont couplés de manière lâche aux **Pods** via les **étiquettes** (labels) et les **sélecteurs** (selectors).

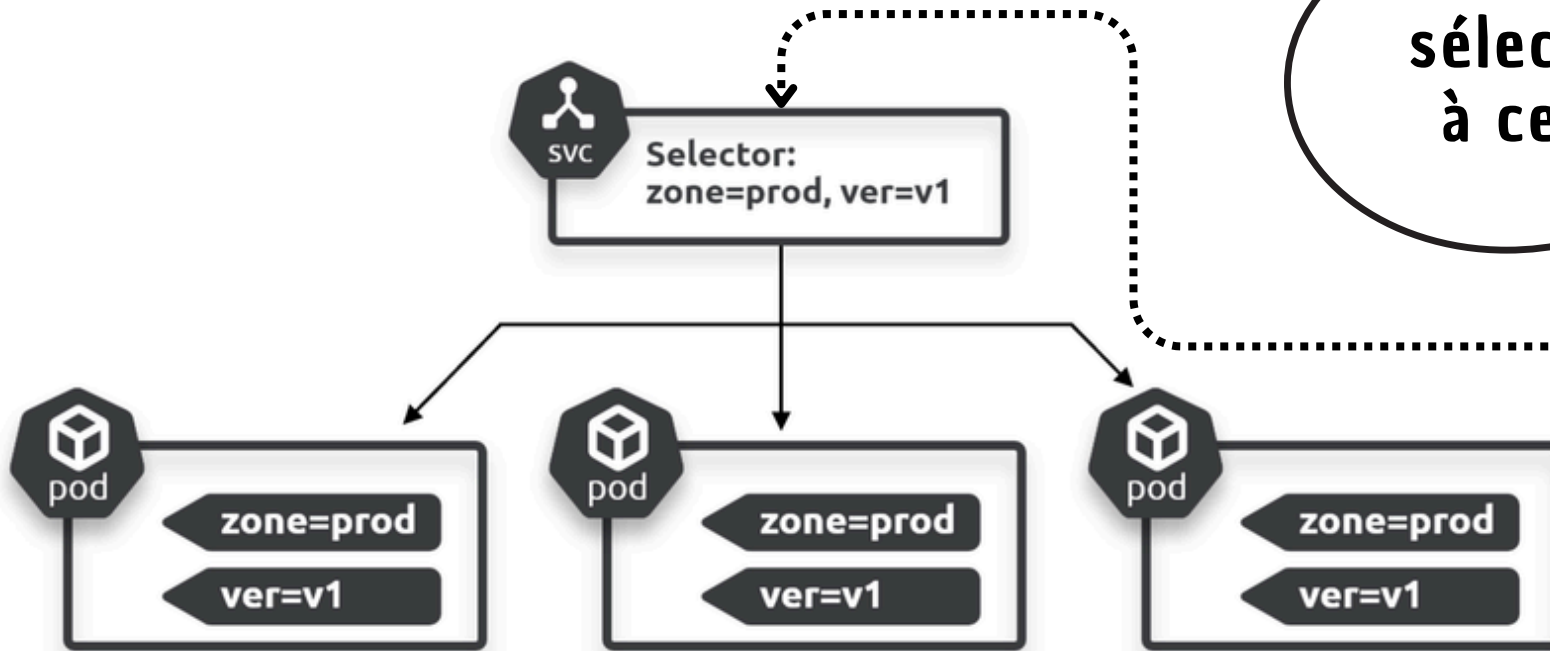
- Même technologie qui couple les Deployments aux Pods.
- Clé de la flexibilité de Kubernetes.

Étiquettes et Couplage Lâche

Exemple de Correspondance

Configuration : 3 Pods avec les étiquettes :

- zone=prod
- version=1



Le Service a un sélecteur correspondant à ces deux étiquettes.

Étiquettes et Couplage Lâche

Résultat

Le **Service** offre une connectivité réseau stable aux trois **Pods**.

- Il répartit la charge (**load-balancing**) entre eux.
- Les requêtes sont envoyées au **Service**, qui les achemine vers les **Pods**.

Règle de Sélection

Pour recevoir du trafic d'un **Service**, un **Pod** doit :

✓ Posséder toutes les étiquettes demandées par le sélecteur du Service.

(Logique booléenne ET)

➡ Il peut aussi avoir des étiquettes supplémentaires que le Service ne recherche pas.

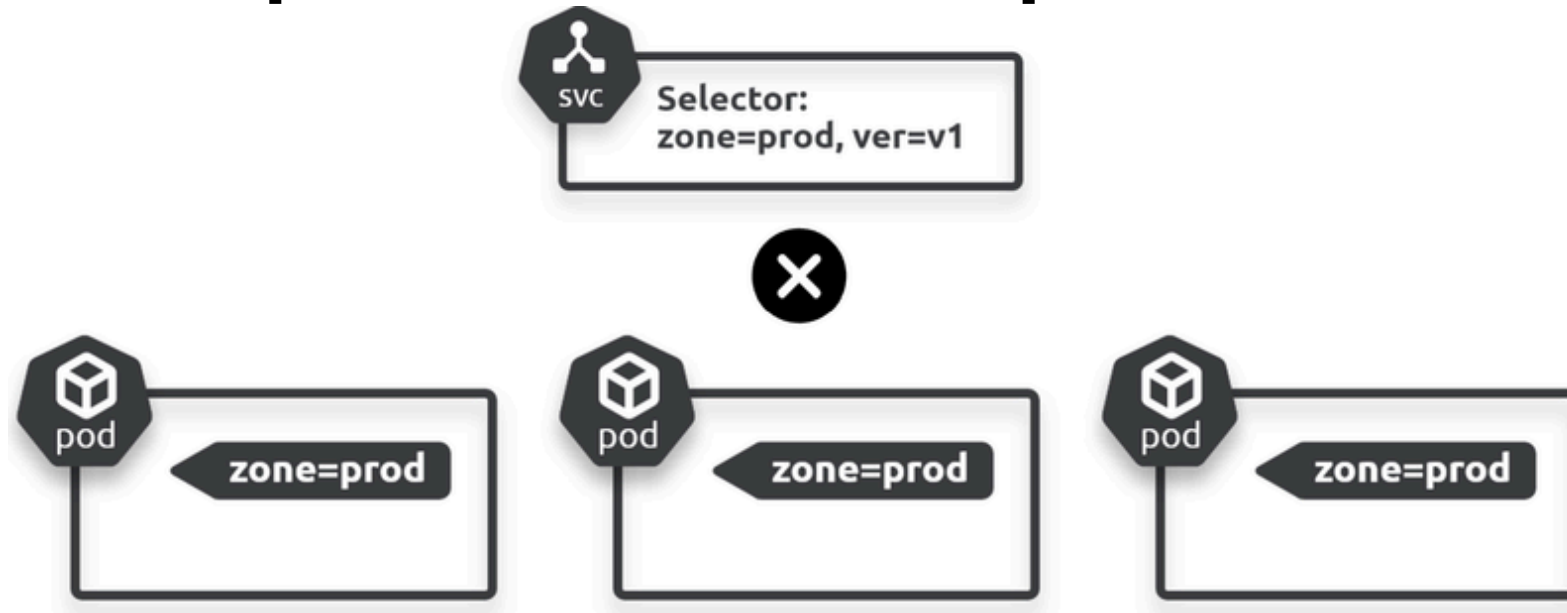
Exemple de Non-correspondance

Situation

Le Service recherche des Pods avec **les deux étiquettes** :

- zone=prod
- version=1

Si aucun Pod ne possède ces deux étiquettes à la fois...



Exemple de Non-correspondance

Situation :

Le Service recherche des Pods avec **les deux étiquettes** :

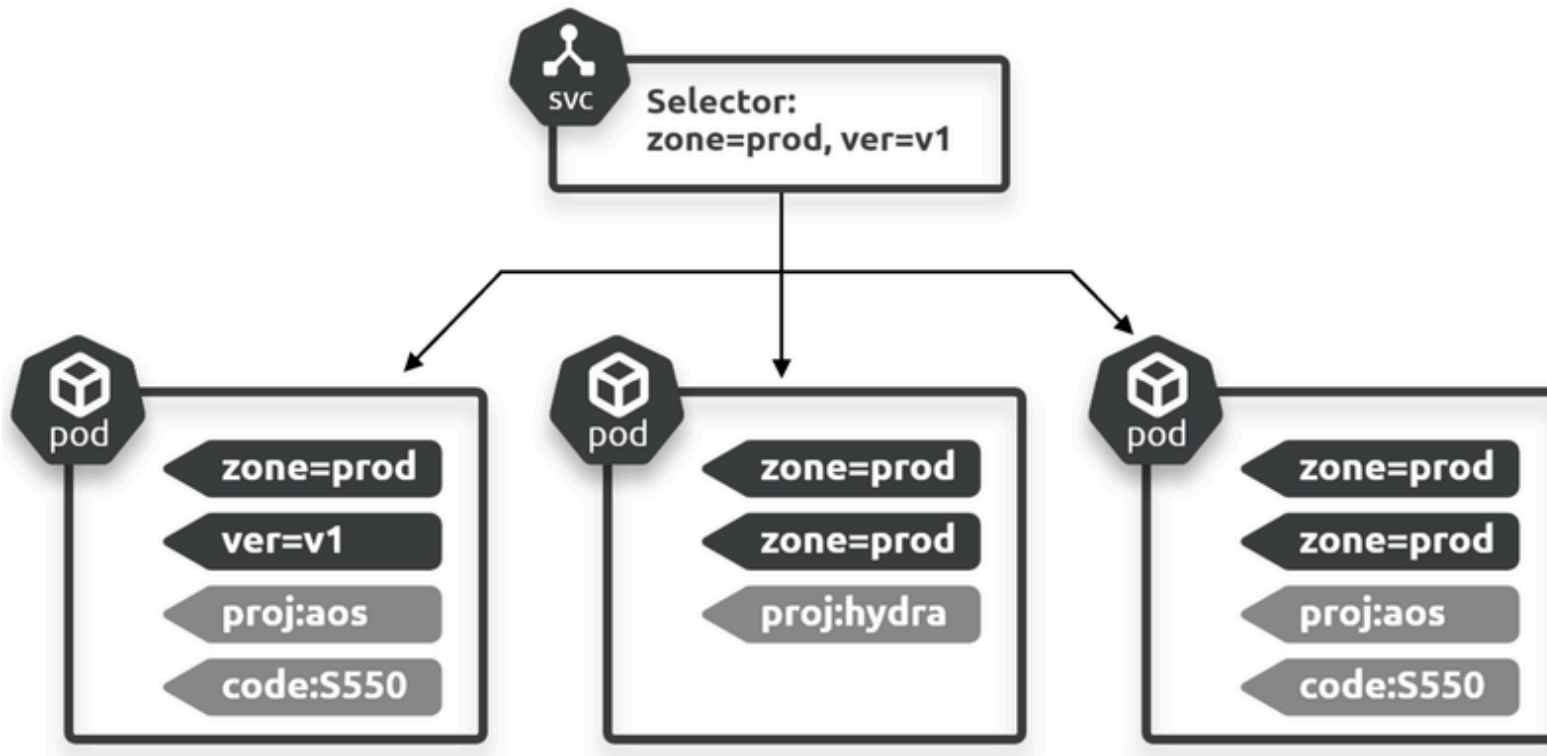
- zone=prod
- version=1

Si aucun Pod ne possède ces deux étiquettes à la fois...

Résultat : **Aucun trafic n'est acheminé.**

Le Service n'a **pas de cible** valide.

Exemple de correspondance réussie



✓ Peu importe les étiquettes supplémentaires sur les Pods, seul compte le fait d'avoir TOUTES les étiquettes demandées.

SCÉNARIO FONCTIONNEL

- Ici, les Pods possèdent toutes les étiquettes requises par le sélecteur du Service.

RÈGLE CLAIRE

1. Le Service recherche deux étiquettes spécifiques.
2. Il trouve les Pods qui les possèdent.
3. Il ignore complètement leurs autres étiquettes supplémentaires.

Labels et selecteurs

```
apiVersion: v1
kind: Service
metadata:
  name: hello-svc
spec:
  ports:
    - port: 8080
  selector:
    app: hello-world
    env: tkb
```

ENVOYER AUX PODS
AVEC CES LABELS
1. hello-world
2. tkb

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 10
  <Extrait...>
  template:
    metadata:
      labels:
        app: hello-world
        env: tkb
    spec:
      containers:
        <Extrait...>
```

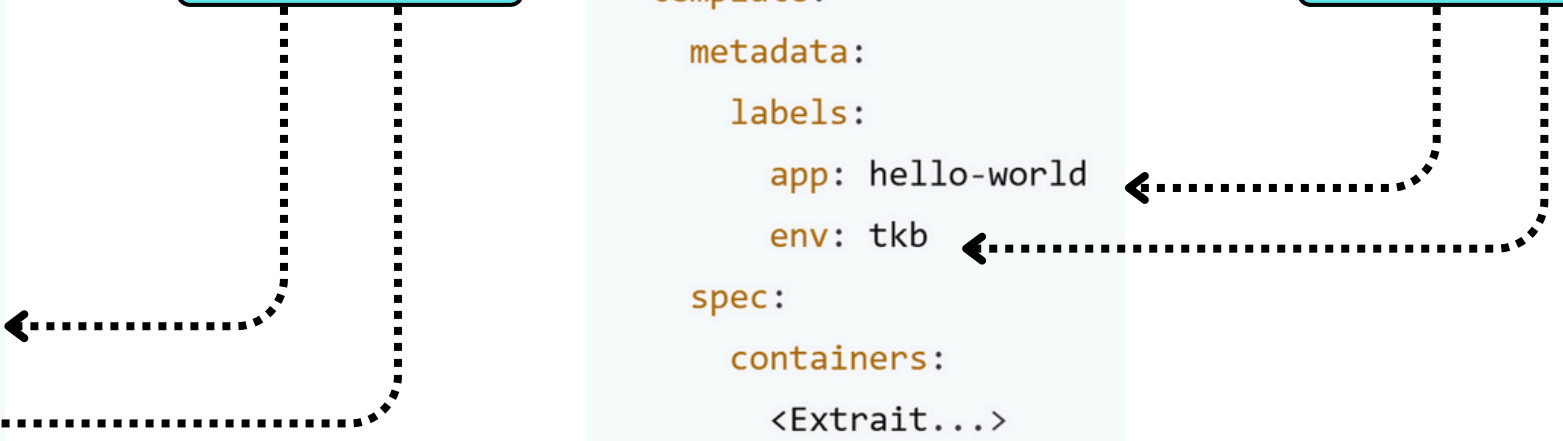
Labels et selecteurs

```
apiVersion: v1
kind: Service
metadata:
  name: hello-svc
spec:
  ports:
    - port: 8080
  selector:
    app: hello-world
    env: tkb
```

ENVOYER AUX
PODS AVEC CES
LABELS
1.hello-world
2.tkb

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 10
  <Extrait...>
  template:
    metadata:
      labels:
        app: hello-world
        env: tkb
    spec:
      containers:
        <Extrait...>
```

Labels du Pod



Labels et selectors

Comment cela fonctionne ?

- Le Service sélectionne les Pods avec les labels
 - app=hello-world
 - env=tkb.
- Le modèle de Pod du Déploiement possède exactement les deux mêmes labels.
- Cela signifie que tous les Pods qu'il déploie correspondront au sélecteur du Service et recevront son trafic.

Ce couplage lâche est la façon dont les Services savent à quels Pods envoyer le trafic.

Services et objets Endpoints

- Lorsque les **Pods** apparaissent et disparaissent (mise à l'échelle, pannes, déploiements, etc.), le **Service** met à jour dynamiquement sa liste de **Pods** sains correspondants.
- Il le fait grâce à une combinaison de sélection par **label** et d'une structure appelée objet **Endpoints**.

Rôle de l'objet Endpoints

Principe

- Chaque fois que vous créez un **Service**, Kubernetes crée automatiquement un objet **Endpoints** associé.
- L'objet **Endpoints** est utilisé pour stocker une liste dynamique de **Pods** sains correspondant au sélecteur de label du **Service**.

Mécanisme

- Kubernetes évalue constamment le sélecteur du **Service** par rapport à tous les **Pods** sains du cluster.
 - Tout nouveau **Pod** correspondant est ajouté à l'objet **Endpoints**.
 - Tout **Pod** qui disparaît est retiré.
- Ainsi, l'objet **Endpoints** est toujours à jour.

Cheminement du trafic

Lors de l'envoi de trafic vers des **Pods** via un **Service** :

- Le DNS interne du cluster résout le nom du **Service** en une adresse IP stable.
- Le trafic est envoyé à cette adresse IP.
- Il est ensuite acheminé vers l'un des **Pods** de la liste des **Endpoints**.

Note : Une application native Kubernetes peut interroger directement l'API **Endpoints**, contournant la résolution DNS et l'utilisation de l'IP du **Service**.

Note sur l'évolution

Note : Les versions récentes de Kubernetes remplacent les objets **Endpoints** par des **Endpoint Slices** plus efficaces.

La fonctionnalité est identique, mais les **Endpoint Slices** offrent de meilleures performances et une plus grande efficacité.

Accès depuis l'intérieur du cluster

Kubernetes prend en charge plusieurs types de Services. Le type par défaut est ClusterIP.

Un Service ClusterIP possède une adresse IP virtuelle stable qui est **accessible uniquement depuis l'intérieur du cluster**.

Nous l'appelons le « ClusterIP ».

Accès depuis l'intérieur du cluster

@Cluster IP

Stable et durable

Cette adresse est programmée dans la structure réseau et garantie stable pour toute la durée de vie du Service.

Découverte de service

Le ClusterIP est enregistré auprès du service DNS interne du cluster sous le nom du Service.

Tous les Pods peuvent l'utiliser

Tous les Pods du cluster sont pré-configurés pour utiliser le service DNS du cluster. Ainsi, tous les Pods peuvent résoudre le nom du Service en ClusterIP.

Exemple de fonctionnement d'un ClusterIP

La création d'un nouveau Service nommé magic-sandbox attribue dynamiquement un ClusterIP stable.

1

Ce nom et ce ClusterIP sont automatiquement enregistrés dans le DNS du cluster.

2

Tous les Pods envoient leurs requêtes de découverte de services au DNS interne.

3

Un Pod peut résoudre magic-sandbox en son ClusterIP.

4

Des règles iptables ou IPVS sont distribuées dans le cluster pour garantir que le trafic envoyé à ce ClusterIP soit routé vers les Pods correspondants.

Résultat : Si un Pod connaît le nom d'un Service, il peut le résoudre en une adresse ClusterIP et se connecter aux Pods derrière ce service.

Remarque : Cela ne fonctionne que depuis l'intérieur du cluster.

Accès depuis l'extérieur du cluster

Kubernetes propose deux types de **Services** pour les requêtes provenant de l'extérieur du cluster :

- **NodePort**
- **LoadBalancer**

Le Service **NodePort** s'appuie sur le type **ClusterIP** et permet un accès externe via un port dédié sur chaque nœud du cluster. Ce port est appelé le « NodePort ».

Exemple de Service NodePort

Exemple de Service NodePort

- **Depuis le cluster** : Les Pods peuvent accéder à ce service via le nom magic-sandbox:8080.
- **Depuis l'extérieur** : Les clients peuvent envoyer du trafic sur le port 30050 de l'adresse IP de n'importe quel nœud du cluster.

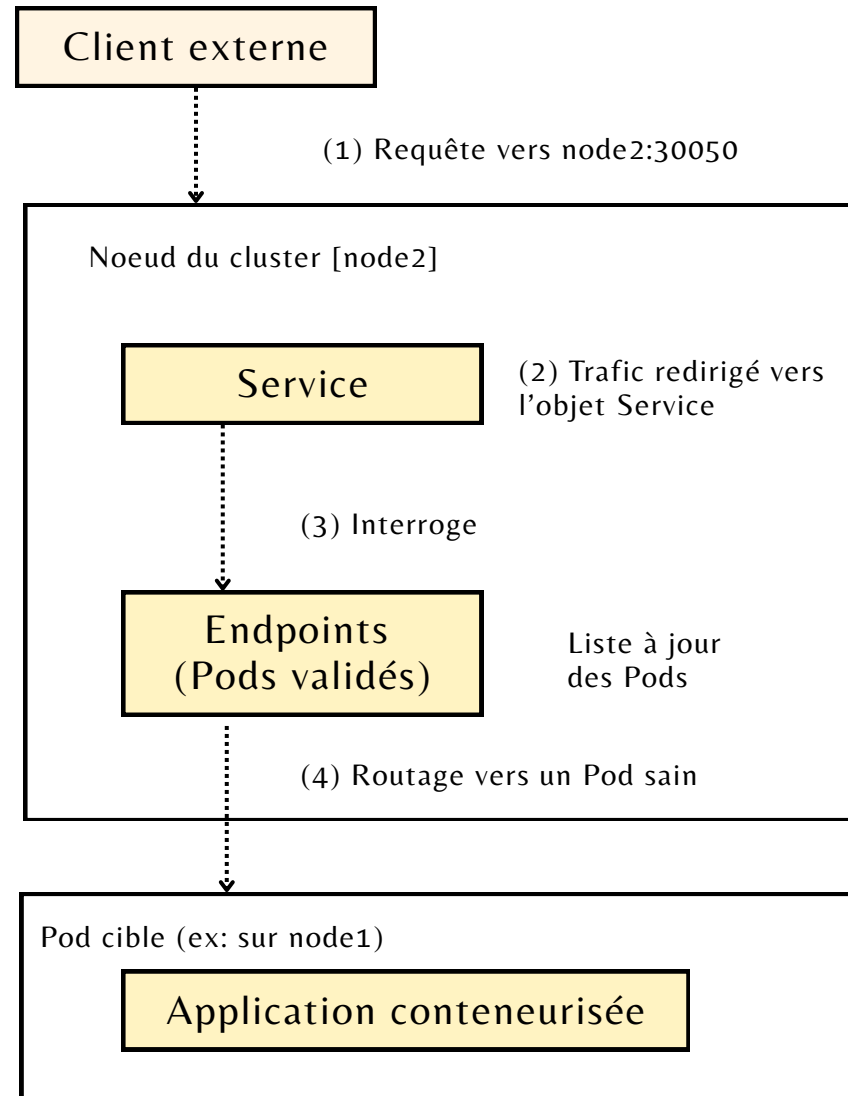
```
apiVersion: v1
kind: Service
metadata:
  name: magic-sandbox
spec:
  type: NodePort
  ports:
    - port: 8080
      nodePort: 30050
  selector:
    app: hello-world
```

Port interne du Service (port ClusterIP)

Port accessible par les clients externes (généralement entre 30000-32767)

Schéma de fonctionnement d'un NodePort

Le Service
effectue un
équilibrage de
charge basique
entre les Pods
correspondants.



Service LoadBalancer

Le Service **LoadBalancer** simplifie encore plus l'accès externe en s'intégrant à un **équilibreur de charge orienté Internet** de votre plateforme cloud sous-jacente.

- **Avantages :**
 - une **adresse IP publique** ou un **nom DNS** haute performance et hautement disponible.
 - accéder directement depuis Internet.
 - enregistrer des noms DNS conviviaux (p.ex., mon-app.mon-domaine.com).
- **Prérequis :** Ne fonctionne que sur les clouds qui le supportent (AWS, GCP, Azure, etc.).

Service LoadBalancer

LoadBalancer Service

Fonction : Simplifie l'accès externe en intégrant un équilibreur de charge Internet-facing fourni par la plateforme cloud.

Avantages :

- Fournit une IP publique ou un nom DNS hautement disponible et performant.
- Permet d'enregistrer des noms DNS conviviaux pour un accès simplifié.
- Pas besoin de connaître les noms ou IPs des nœuds du cluster.

Limitations :

- Fonctionne uniquement sur les plateformes cloud qui les supportent.



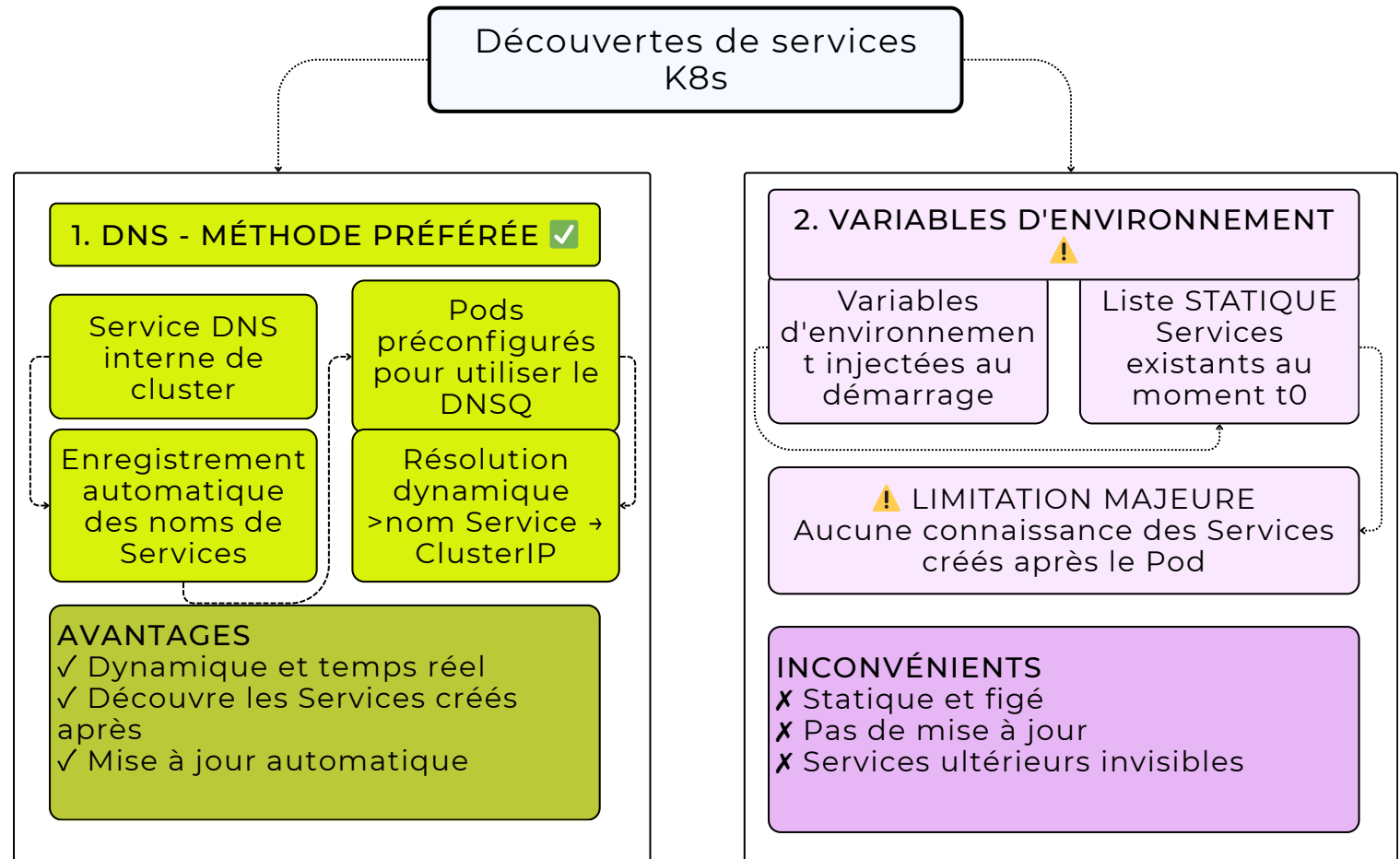
Utilisation pratique :

- Créez et utilisez un LoadBalancer Service dans la section pratique.

Découverte de Service

Kubernetes implémente la découverte de services de deux manières principales :

1. DNS (méthode préférée)
2. Variables d'environnement (déconseillée)



Méthode impérative

Méthode impérative : Non “Kubernetes”

Elle désynchronise votre cluster et vos applications par rapport à vos fichiers YAML.

Première étape (déclarative) :

Déployons le déploiement de manière déclarative.

```
$ kubectl apply -f deploy.yml  
deployment.apps/svc-test created
```

Création impérative d'un Service

Maintenant, créons impérativement un Service pour ce déploiement.

```
$ kubectl expose deployment svc-test --type=NodePort  
service/hello-svc exposed
```

Cette commande crée un Service qui fournira la mise en réseau et l'équilibrage de charge pour les Pods du déploiement svc-test.

Vérification du Service créé

Vérifions la création du Service :

```
$ kubectl get svc -o wide
```

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE	SELECTOR
kubernetes	ClusterIP	10.43.0.1	443/TCP	19d	<none>
svc-test	NodePort	10.43.48.145	8080:30013/TCP	10s	chapter=services

Note : La commande expose a intelligemment :

- lu la spécification du déploiement
- configuré le bon sélecteur de labels

- utilisé le bon port de conteneur
- donné au Service le même nom que le déploiement

Détails du Service

Taper la commande !

```
$ kubectl describe svc svc-test
```

```
Name:          svc-test
Namespace:     default
Selector:      chapter=services      # Labels recherchés
Type:          NodePort
IP:            10.43.48.145          # ClusterIP permanent (VIP)
Port:          <unset> 8080/TCP       # Port d'écoute dans le cluster
TargetPort:    8080/TCP              # Port d'écoute de l'application
NodePort:      <unset> 30013/TCP      # Port pour accès externe
Endpoints:     10.42.0.116:8080,...  # Liste dynamique des Pods sains
```

Accès à l'application via NodePort

Avec le NodePort (30013), accédons à l'application via un navigateur web.

Prérequis :

- IP d'au moins un nœud du cluster
- Accessibilité depuis votre navigateur (IP routable publiquement si via Internet)

Exemple d'accès :

```
http://54.246.255.52:30013
```

Architecture de l'application déployée

Flux de trafic : Navigateur Web → Service Kubernetes → Pods d'application

1. Navigateur Web

- Envoie une requête sur le port 30013.

2. Nœud du Cluster (IP: 54.246.255.52)

- Reçoit la requête et redirige le trafic.

3. Service Kubernetes (svc-test)

- Reçoit le trafic et effectue l'équilibrage de charge.

4. Pods d'application

- Reçoivent le trafic et traitent la requête (écoutent sur le port 8080).

Architecture de l'application déployée

Flux de trafic : Navigateur Web → Service Kubernetes → Pods d'application

1. Navigateur Web

- Envoie une requête sur le port 30013.

2. Nœud du Cluster (IP: 54.246.255.52)

- Reçoit la requête et redirige le trafic.

3. Service Kubernetes (svc-test)

- Reçoit le trafic et effectue l'équilibrage de charge.

4. Pods d'application

- Reçoivent le trafic et traitent la requête (écoutent sur le port 8080).

Caractéristiques :

- Application web simple écoutant sur le port 8080
- **Service** Kubernetes mappe le port 30013 vers le port 8080
- Les **NodePorts** dynamiquement attribués entre 30 000 et 32 767
- Possibilité de spécifier un port explicitement via YAML

Nettoyage pour la suite

Avant de voir la méthode déclarative (la bonne façon), supprimons le Service créé impérativement.

```
$ kubectl delete svc svc-test  
service "svc-test" deleted
```

À suivre : La manière propre et recommandée - la méthode déclarative avec YAML.

Méthode déclarative

La manière déclarative (la bonne façon)

Il est temps de faire les choses correctement... la "manière Kubernetes".
Fichier manifeste d'un Service :

```
apiVersion: v1
kind: Service
metadata:
  name: svc-test
spec:
  type: NodePort
  ports:
    - port: 8080
      nodePort: 30001
      targetPort: 8080
      protocol: TCP
  selector:
    chapter: services
```

Port externe sur tous les nœuds

Port d'écoute des Pods

Sélectionne les Pods avec ce label



Explication du manifeste YAML

Section	Explication
<code>apiVersion: v1</code>	Objet mature du groupe API core
<code>kind: Service</code>	Définition d'un objet Service
<code>metadata.name</code>	Nom du Service (peut avoir labels/annotations)
<code>spec.type: NodePort</code>	Type de Service pour accès externe
<code>spec.ports.port</code>	Port d'écoute interne (8080)
<code>spec.ports.nodePort</code>	Port externe sur tous les nœuds (30001)
<code>spec.ports.targetPort</code>	Port des Pods cibles (8080)
<code>spec.selector</code>	Sélecteur de labels pour trouver les Pods

```
apiVersion: v1
kind: Service
metadata:
  name: svc-test
spec:
  type: NodePort
  ports:
    - port: 8080
      nodePort: 30001
      targetPort: 8080
      protocol: TCP
  selector:
    chapter: services
```

Déploiement déclaratif

Déployons le Service avec kubectl apply :

```
$ kubectl apply -f svc.yml  
service/svc-test created
```

Principe clé : Le fichier YAML est la **source de vérité**.

- Versionné dans Git
- Documenté
- Reproductible

Inspection du Service

Vérifions le Service déployé :

```
$ kubectl get svc svc-test
```

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
svc-test	NodePort	100.70.40.2	8080:30001/TCP	8s

Accès à l'application :

- Interne : svc-test:8080 (depuis le cluster)
- Externe : [IP_DU_NOEUD]:30001 (depuis internet)

Assurez-vous que les règles de firewall autorisent le trafic.

Objets Endpoints et EndpointSlices

Rappel : Chaque Service a un objet Endpoints (ou EndpointSlices) du même nom.
EndpointSlices (plus moderne et performant) :

```
$ kubectl get endpointslices
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
svc-test-sbhbj	IPv4	8080	10.42.1.119, 10.42.0.117...	6m38s

Détail d'un EndpointSlice

```
$ kubectl describe endpointslice svc-test-sbhbj
```

AddressType: IPv4

Ports:

Name	Port	Protocol
----	----	-----
<unset>	8080	TCP

Endpoints:

- Addresses: 10.42.1.119

Conditions:

Ready: true

TargetRef: Pod/svc-test-84db6ff656-wd5w7 # Référence au Pod

Topology: kubernetes.io/hostname=k3d-gsk-book-server-0

- Addresses: 10.42.0.117

...

Points clés des Endpoints

Contenu d'un EndpointSlice :

- Liste dynamique des Pods sains correspondant au sélecteur
- Une entrée par Pod avec :
 - Adresse IP du Pod
 - État Ready
 - Référence au Pod (TargetRef)
 - Topologie (nœud hébergeant le Pod)

Avantage : Mise à jour automatique quand les Pods changent (scale, crash, rollout...).

Comparaison des approches

Méthode	Avantages	Inconvénients
Impérative (expose)	Rapide pour le test	Pas de source de vérité versionnée
Déclarative (YAML)	Recommandée Versionnable Documentée Reproductible	Nécessite un fichier YAML

Bonnes pratiques : Toujours utiliser la méthode déclarative pour la production.

Le Load Balancer

Service LoadBalancer

Meilleur type de Service :

C'est aussi le plus simple !

Si votre cluster est sur une plateforme cloud, déployer un Service avec **type=LoadBalancer** va :

- Provisionner un équilibreur de charge orienté Internet de votre cloud
- Le configurer pour envoyer du trafic vers votre Service
- Vous donner une adresse IP publique ou un nom DNS stable

Manifeste YAML pour un LoadBalancer

Fonctionnement :

- L'équilibreur de charge cloud écoute sur le port **9000**
- Il achemine le trafic vers les Pods sur le port **8080**
- Il cible les Pods avec le label **chapter=services**

```
apiVersion: v1
kind: Service
metadata:
  name: cloud-lb
spec:
  type: LoadBalancer
  ports:
  - port: 9000
    targetPort: 8080
  selector:
    chapter: services
```

Type
LoadBalancer

Port externe
de l'équilibreur
de charge

Port
d'écoute des
Pods

Même
sélecteur que
les Services
précédents

Déploiement du LoadBalancer

```
$ kubectl apply -f lb.yml  
service/cloud-lb created
```

Vérifions l'état avec --watch :

```
$ kubectl get svc --watch
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
cloud-lb	LoadBalancer	10.43.128.113	172.21.0.4	9000:32688/TCP	47s

Caractéristiques du LoadBalancer

La colonne EXTERNAL-IP :

- Adresse IP publique attribuée par le cloud
- Peut être un nom DNS sur certaines plateformes
- Peut prendre 1-2 minutes à apparaître (temps de provisionnement)

Processus en arrière-plan :

1. Kubernetes envoie une requête à l'API du cloud
2. Le cloud provisionne un équilibreur de charge
3. Configuration automatique des règles de routage
4. Attribution de l'adresse IP/DNS publique

Accès à l'application

URL d'accès

```
http://172.21.0.4:9000
```

Avantages :

-  Haute disponibilité : Gérée par le cloud provider
-  Haute performance : Infrastructure cloud optimisée
-  Évolutivité automatique
-  Sécurité intégrée (certificats SSL, WAF...)
-  Monitoring natif du cloud

Comparaison des types de Service

Type	Usage	Accès externe	Infrastructure
ClusterIP	Interne au cluster	✗ Non	Kubernetes uniquement
NodePort	Développement/test	✓ Port fixe sur tous les nœuds	Kubernetes + réseau
LoadBalancer	Production cloud	✓ IP/DNS publique	Kubernetes + Cloud LB

Nettoyage du laboratoire

Suppression de toutes les ressources :

```
$ kubectl delete -f deploy.yml -f svc.yml -f lb.yml  
deployment.apps "svc-test" deleted  
service "svc-test" deleted  
service "cloud-lb" deleted
```

Note : Les objets Endpoints/Endpointslices sont automatiquement supprimés avec leur Service.

Fonctionnalités automatiques supprimées :

- L'équilibreur de charge cloud (désalloué)
- Les règles de firewall cloud
- Les adresses IP publiques (libérées)

Résumé des Services Kubernetes

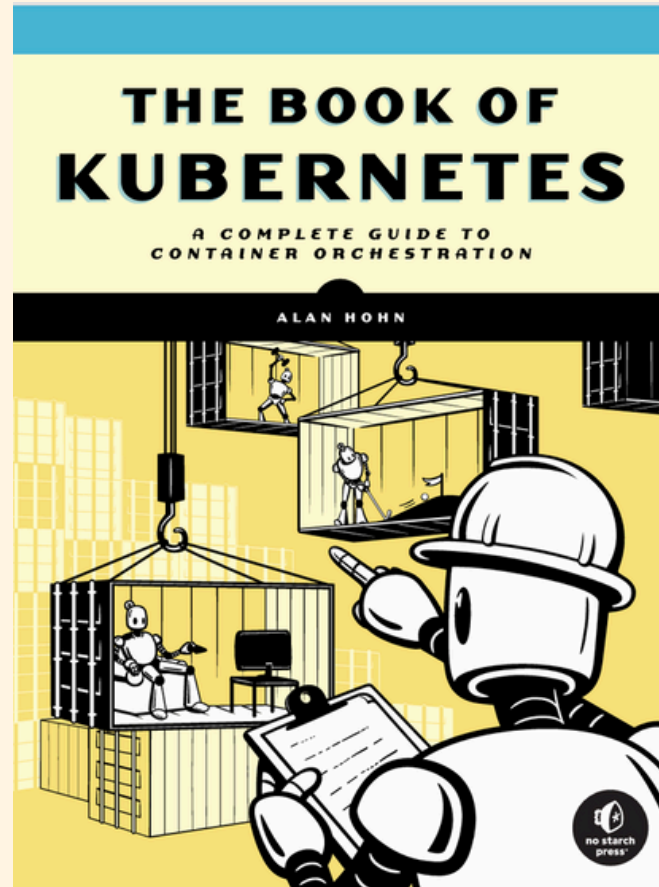
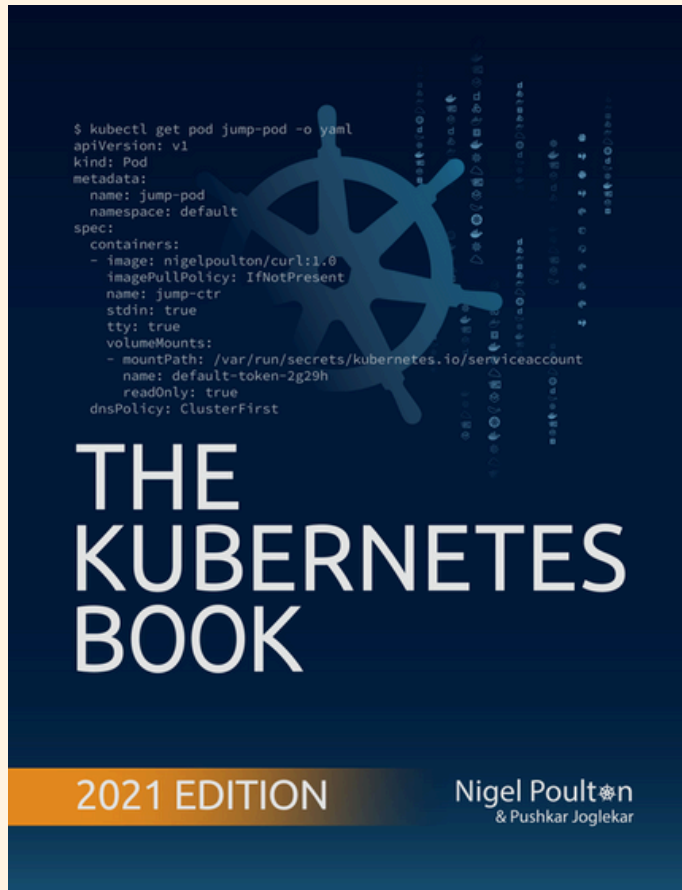
Types principaux :

1. ClusterIP : Communication interne (microservices)
2. NodePort : Accès externe simple (ports fixes)
3. LoadBalancer : Production cloud (LB managé)

Bonnes pratiques :

- Utiliser LoadBalancer pour les applications web publiques
- Utiliser ClusterIP pour les communications internes
- Toujours privilégier la méthode déclarative (YAML)
- Versionner les manifests dans Git

Documentation



Kubernetes Official Website Documentation

Kubernetes Documentation

Kubernetes is an open source container orchestration engine for automating deployment, scaling, and management of containerized applications. The open source project is hosted by the Cloud Native...

 [Kubernetes](#)

